IC TRAINING CENTER VIET NAM

FUNDAMENTAL IC DESIGN AND VERIFICATION COURSE

FINAL PROJECT

TIMER IP DESIGN SPECIFICATION

Student: Pham Tran Dang Minh

Class: IC28

Instructor: Kha At Kham

Contents

LIST OF TABLES 3
LIST OF FIGURES 4
1.Overview 5
1.1. Introduction 5
1.2. Main features 5
1.3. Block diagram 6
1.4. Interface signals 7
2.Register Specification 8
2.1. Register Summary 8
2.2. Timer Control Register (TCR) 8
2.3. Timer Data Register 0 (TDR0) 10
2.4. Timer Data Register 1 (TDR1) 10
2.5. Timer Compare Register 0 (TCMP0) 11
2.6. Timer Compare Register 1 (TCMP1) 11
2.7. Timer Interrupt Enable Register (TIER) 12
2.8. Timer Interrupt Status Register (TISR) 13
2.9. Timer Halt Control Status Register (THCSR) 13
3. Functional Description 14
3.1. APB slave / Register 14
3.2. Counter Control Combines With Halted, And Counting Mode 23
3.3. Timer Interrupt 34
4. Report 34

LIST OF TABLES

Table 1: Interface signals of timer IP 7
Table 2: Register Summary 8
Table 3. Timer Control Register (TCR).....9
Table 4. Timer Data Register 0 (TDR0) 10
Table 5. Timer Data Register 1 (TDR1) 11
Table 6. Timer Compare Register 0 (TCMP0) 11
Table 7. Timer Compare Register 1 (TCMP1) 12
Table 8. Timer Interrupt Enable Register (TIER).....12
Table 9. Timer Interrupt Status Register (TIER) 13
Table 10. Timer Halt Control Status Register (THCSR) 14
Table 11. V-plan for verification 35

LIST OF FIGURES

Figure 1: block diagram of Timer IP 6
Figure 2. APB Slave timing 15
Figure 3. APB slave logic diagram 16
Figure 4. TCR Register logic diagram 17
Figure 5. TDR0 Register logic diagram 18
Figure 6. TDR1 Register logic diagram 19
Figure 7. TCMP0 Register logic diagram 20
Figure 8. TCMP1 Register logic diagram 21
Figure 9. TIER and TISR Register logic diagram 22
Figure 10. THCSR Register logic diagram 23
Figure 11. Counter Control logic diagram 24
Figure 12. Waveform of the counter counting in default mode 25
Figure 13. Waveform of the counter counting in mode div_val = 0 26
Figure 14. Waveform of the counter counting in mode div_val = 1 27
Figure 15. Waveform of the counter counting in mode div_val = 2 28
Figure 16. Waveform of the counter counting in mode div_val = 3 29
Figure 17. Waveform of the counter counting in mode div_val = 4 30
Figure 18. Waveform of the counter counting in mode div_val = 5 31
Figure 19. Waveform of the counter counting in mode div_val = 6 32
Figure 20. Waveform of the counter counting in mode div_val = 7 33
Figure 21. Waveform of the counter counting in mode div_val = 8 34
Figure 22. Testcases list 39
Figure 23. Test Cases Report 40
Figure 24. Coverage 41

1.Overview

1.1. Introduction

Timer is an essential module for every chip. This is used to generate accurate timing interval or controlling the timing of various operations within the circuit.

Timer can be used in various application: pulse generation, delay generation, event generation, PWM generation, Interrupt generation ….

In this project, a timer module is customized from CLINT module of industrial RISC-V architecture. It is used to generate interrupt based on user settings.

1.2. Main features

- 64-bit count-up
- 12-bit address
- Register set is configured via APB bus (IP is APB slave).
- Support wait state (1 cycle is enough) and error handling
- Support byte access
- Support halt (stop) in debug mode
- Timer uses active low async reset.
- Counter can be counted based on system clock or divided up to 256.
- Support timer interrupt (can be enabled or disabled).

1.3. Block diagram

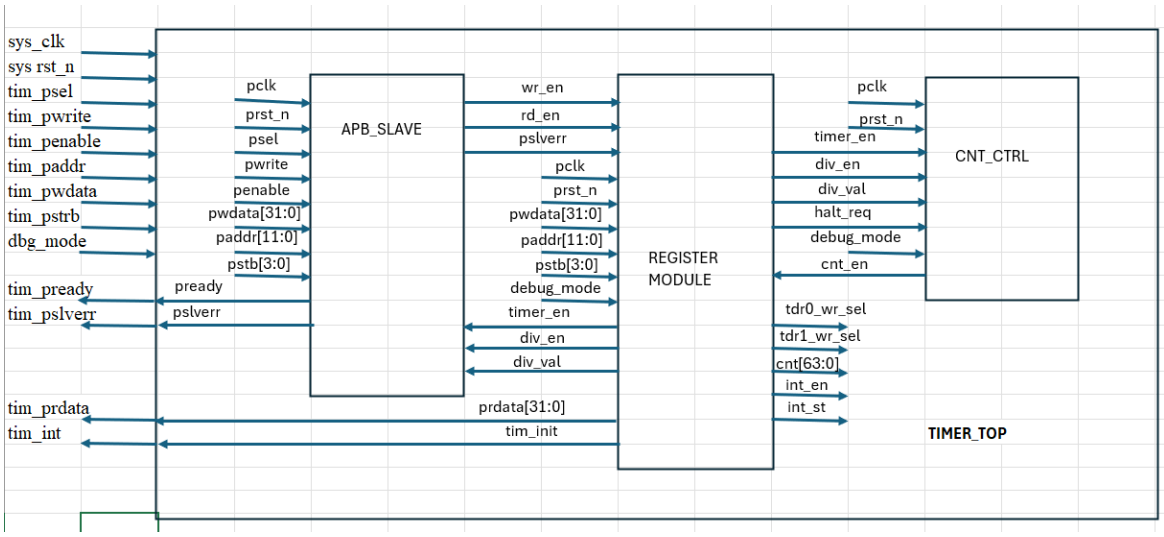

Figure 1: block diagram of Timer IP

- **APB Slave Block**
  The APB Slave block implements the APB protocol interface, handling bus transactions from the system bus. It performs address decoding based on `paddr`, controls read and write operations using `psel`, `penable`, and `pwrite`, and accepts write data through `pwdata`. The block generates internal control signals `wr_en` and `rd_en` to indicate valid write and read accesses to the Register Module. It also returns transaction responses to the APB master by asserting `pready` and `pslverr`, and forwards read data through `prdata`.
- **Register Module**
  The Register Module provides a programmable register interface for the timer through the APB Slave. It contains all timer-related configuration and status registers, including the Timer Control Register (TCR), Timer Data Registers (TDR0, TDR1), Timer Compare Registers (TCMP0, TCMP1), Timer Interrupt Enable Register (TIER), Timer Interrupt Status Register (TISR), and Timer Halt Control Status Register (THCSR).
  This block stores the 64-bit counter value (`cnt[63:0]`) and manages register read/write operations based on `wr_en` and `rd_en`. It generates control signals such as `timer_en`, `div_en`, and `div_val` for the Counter Control block. In addition, it processes `halt_req` and `debug_mode` signals to support conditional counter halting and debug operation. The Register Module also generates interrupt-related signals `int_en` and `int_st`, and provides write-select signals (`tdr0_wr_sel`, `tdr1_wr_sel`) for counter loading.

- **Counter Control (CNT_CTRL) Block**
  The Counter Control block is responsible for determining when the timer counter is allowed to increment. It receives control inputs including `timer_en`, `div_en`, `div_val`, `debug_mode`, and `halt_req`. Based on these signals, it generates the `cnt_en` signal, which enables or disables the counter update within the Register Module. This block ensures that the counter operates correctly under normal, divided-clock, halted, or debug conditions.
- **Interrupt Logic**
  The Interrupt logic monitors the interrupt enable signal (`int_en`) and interrupt status signal (`int_st`) generated by the Register Module. When a match event occurs and the corresponding interrupt is enabled, the block asserts the `tim_int` signal to notify the system of a timer interrupt. The interrupt status can be cleared or updated through register writes via the APB interface.
- **TIMER_TOP Module**
  The TIMER_TOP module integrates the APB Slave, Register Module, Counter Control block, and Interrupt logic into a complete timer IP. It connects the APB bus interface to internal timer logic and exposes system-level signals such as `sys_clk`, `sys_rst_n`, `tim_prdata`, `tim_pready`, `tim_pslverr`, and `tim_int`.

1.4. Interface signals

Table 1: Interface signals of timer IP

Signal name	Width	Direction	Description
sys_clk	1	Input	**System clock** used for all sequential logic in the timer.
sys_rst_n	1	Input	**Active-low system reset** that initializes all internal timer logic.
tim_psel	1	Input	**APB select signal**, asserted when the timer IP is selected for an APB transfer
tim_pwrite	1	Input	**APB write control signal**: 1 indicates write access, 0 indicates read access.
tim_penable	1	Input	**APB enable signal**, asserted during the access phase of an APB transfer.
tim_paddr[11:0]	12	Input	**APB address bus** used to select internal timer registers.
tim_pwdata[31:0]	32	Input	**APB write data bus** carrying data written to timer registers.
tim_pstrb[3:0]	4	Input	**APB write strobe signals** used to indicate valid byte lanes during write accesses
dbg_mode	1	Input	**Debug mode control signal**, which allows the counter to halt based on

			halt_req. *(Note: The testbench must not change dbg_mode after timer_en is asserted.)*
tim_prdata[31:0]	32	Output	**APB read data bus** returning data from timer registers.
tim_pready	1	Output	**APB ready signal** indicating completion of the current APB transfer
tim_pslverr	1	Output	**APB slave error signal**, asserted on invalid or prohibited accesses.
tim_int	1	Output	**Timer interrupt output**, asserted when an interrupt condition is detected.

2.Register Specification

2.1. Register Summary

Table 2: Register Summary

Address	Item	Register name
0x00	TCR	Timer Control Register
0x04	TDR0	Timer Data Register 0
0x08	TDR1	Timer Data Register 1
0x0C	TCMP0	Timer Compare Register 0
0x10	TCMP1	Timer Compare Register 1
0x14	TIER	Timer Interrupt Enable Register
0x18	TISR	Timer Interrupt Status Register
0x1C	THCSR	Timer Halt Control Status Register
Others	Reserved	

2.2. Timer Control Register (TCR)

Offset address: 0x0

Reset value: 0x0000_0100

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				DIV_VAL[3:0]				Reserved						DIV_EN	TIM_EN
						rw								rw	rw

Table 3. Timer Control Register (TCR)

Bit	Name	Type	Default value	Description
31:12	Reserved	-	20’h0	Reserved
11:8	DIV_VAL[3:0]	RW	4’b0001	Counter control mode setting: • 4’b0000: Counting speed is not divided • 4’b0001: Counting speed is divided by 2 (default) • 4’b0010: Counting speed is divided by 4 • 4’b0011: Counting speed is divided by 8 • 4’b0100: Counting speed is divided by 16 • 4’b0101: Counting speed is divided by 32 • 4’b0110: Counting speed is divided by 64 • 4’b0111: Counting speed is divided by 128 • 4’b1000: Counting speed is divided by 256 • Others: reserved - prohibit settings. ▪ When setting the prohibit value, div_val is not changed. ▪ Note: user must not change div_val while timer_en is High ▪ Add hardware logic to ensure div_val is prohibited to change when timer_en is High. Access is error response in this case when setting prohibit value to div_val
7:2	Reserved	RO	6’h0	Reserved
1	DIV_EN	RW	1’b0	Counter control mode enable. • 0: Disabled. Counter counts with normal speed based on system clock • 1: Enabled. The coutingspeed of counter is controlled based on div_val ▪ Note: user must not change div_en while timer_en is High ▪ Add hardware logic to ensure div_en is prohibited to change when timer_en is

				High. Access is error response in this case.
0	TIM_EN	RW	1'b0	Timer enable • 0: Disabled. Counter does not count. • 1: Enabled. Counter starts counting. Timer_en changes from H->L will initialize the TDR0/1 to their initial value

2.3. Timer Data Register 0 (TDR0)

Offset address: 0x4

Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TDR0[31:16] - rw															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TDR0[15:0] - rw															

Table 4. Timer Data Register 0 (TDR0)

Bit	Name	Type	Default value	Description
31:0	TDR0	RW	32'h0000_0000	Lower 32-bit of 64-bit counter. Value of this register is cleared to initial value when timer_en changes from H->L

2.4. Timer Data Register 1 (TDR1)

Offset address: 0x8

Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TDR1[31:16] - rw															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TDR1[15:0] - rw															

Table 5. Timer Data Register 1 (TDR1)

Bit	Name	Type	Default value	Description
31:0	TDR1	RW	32'h0000_0000	Upper 32-bit of 64-bit counter. Value of this register is cleared to initial value when timer_en changes from H->L

2.5. Timer Compare Register 0 (TCMP0)

Offset address: 0xC

Reset value: 0xffff_ffff

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TCMP0[31:16] - rw															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TCMP0[15:0] - rw															

Table 6. Timer Compare Register 0 (TCMP0)

Bit	Name	Type	Default value	Description
31:0	TCMP0	RW	32'hffff_ffff	Lower 32-bit of 64-bit compare value. Interrupt pending bit is asserted when counter value is qual to compare value.

2.6. Timer Compare Register 1 (TCMP1)

Offset address: 0x10

Reset value: 0xffff_ffff

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TCMP1[31:16] - rw															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TCMP1[15:0] - rw															

Table 7. Timer Compare Register 1 (TCMP1)

Bit	Name	Type	Default value	Description
31:0	TCMP1	RW	32'hffff_ffff	Upper 32-bit of 64-bit compare value. Interrupt pending bit is asserted when counter value is qual to compare value.

2.7. Timer Interrupt Enable Register (TIER)

Offset address: 0x14

Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															int_en
															rw

Table 8. Timer Interrupt Enable Register (TIER)

Bit	Name	Type	Default value	Description
31:1	Resered	RO	31'h0	Reserved
0	int_en	RW	1'b0	Timer interrupt enable 0: Timer interrupt is disabled. 1: Timer interrupt is enabled. ▪ When this bit is 0, no timer interrupt is output. ▪ When this bit is 1, timer interrupt pending bit can be output (interrupt pending bit is set when counter value is equals to compare value). ▪ Clearing this bit to 0 while interrupt is asserting will mask the interrupt to 0 but does not affect the interrupt pending bit (TISR.int_st bit).

2.8. Timer Interrupt Status Register (TISR)

Offset address: 0x18

Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															int_st
															rw

Table 9. Timer Interrupt Status Register (TIER)

Bit	Name	Type	Default value	Description
31:1	Resered	RO	31’h0	Reserved
0	int_st	RW1C	1’b0	Timer interrupt trigger condition status bit (interrupt pending bit) 0: the interrupt trigger condition does not occur. 1: the interrupt trigger condition occurred. Write 1 when this bit is 1 to clear it to 0. Write 0 when this bit is 1 has no effect Write to this bit when it is 0 has no effect. Note1: When interrupt trigger condition occurred (counter reached compare value), counter continues to count normally. Note2: the clearance condition has highest priority.

2.9. Timer Halt Control Status Register (THCSR)

Offset address: 0x1C

Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16

Reserved

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														halt_ack	halt_req
														RO	rw

Table 10. Timer Halt Control Status Register (THCSR)

Bit	Name	Type	Default value	Description
31:2	Resered	RO	31'h0	Reserved
1	halt_ack	RO	1'b0	Timer halt acknowledge 0: timer is NOT halted 1: timer is halted Timer accepts the halt request only in debug mode, indicates by debug_mode input signal
0	halt_req	RW	1'b0	Timer halt request 0: no halt req. 1: timer is requested to halt.

3. Functional Description

3.1. APB slave / Register

Read/write to reserved area is RAZ/WI (read as zero, write ignored)

o Support APB 32-bit transfer

o Support byte access: bus can access to individual bytes in the register.

o Support wait state (1cycle) to improve the timing.

o Support error handling for some prohibited access:

- Write prohibited value to TCR.div_val
- div_enor div_val changes during timer is operating

o When error occurs, data is not written into register bit/fields

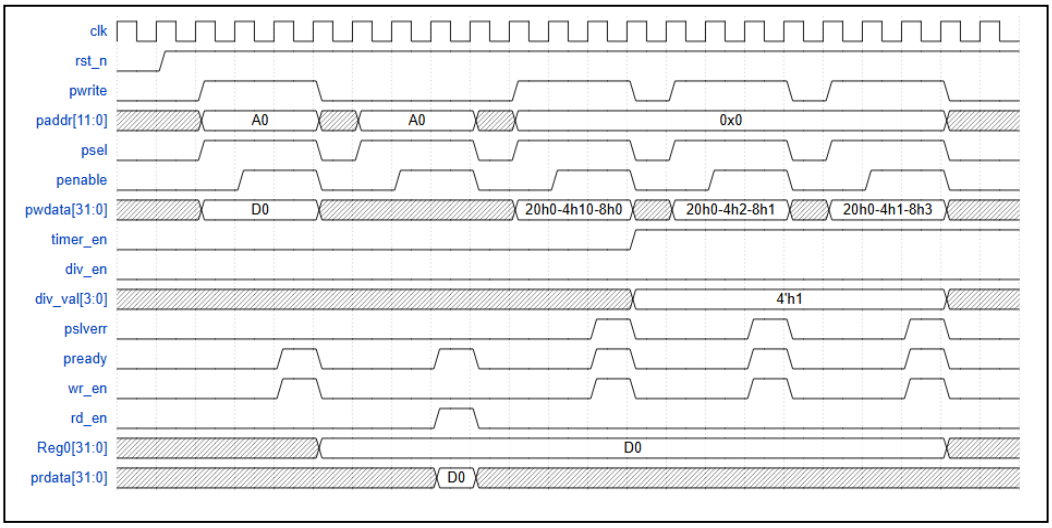

Figure 2. APB Slave timing

wr_en is asserted one cycle after psel, pwrite, and penable are all high, and it de-asserts after remaining high for one cycle. This signal is sent to the register module to store data into the registers.

rd_en is asserted one cycle after pwrite is Low, psel and penable are high, and it de-asserts after remaining high for one cycle. This signal is sent to the Register block to trigger a read operation and return data to the APB interface.

pready indicates that the APB slave has completed the current read or write transfer.

Reg0 is a register that holds its updated value after a write operation.

prdata outputs the read data when rd_en is High.

psel, penable, pwrite, pwdata, and paddr remain stable and do not change during an active APB transfer.

pslverr indicates an error response when an invalid value is written to div_val, or when div_en/div_val are modified while timer_en is High.

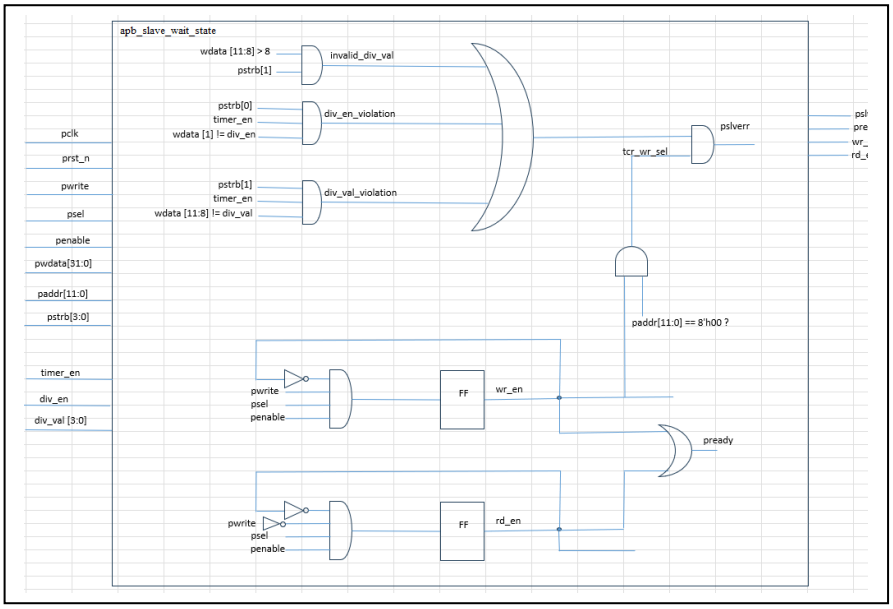

Figure 3. APB slave logic diagram

- wr_en is generated by a flip-flop and is asserted for one clock cycle when psel, penable, and pwrite are High, indicating a valid write access.
- rd_en is generated by a flip-flop and is asserted for one clock cycle when psel and penable are High while pwrite is Low, indicating a valid read access.
- pready is asserted whenever either wr_en or rd_en is High, signaling the completion of the APB transfer.
- pstrb[1] and pstrb[0] indicate active byte lanes during write accesses and support byte-level register updates.

- pslverr is asserted when an illegal write occurs, such as writing a value greater than 8 to div_val, or modifying div_en or div_val while timer_en is High.

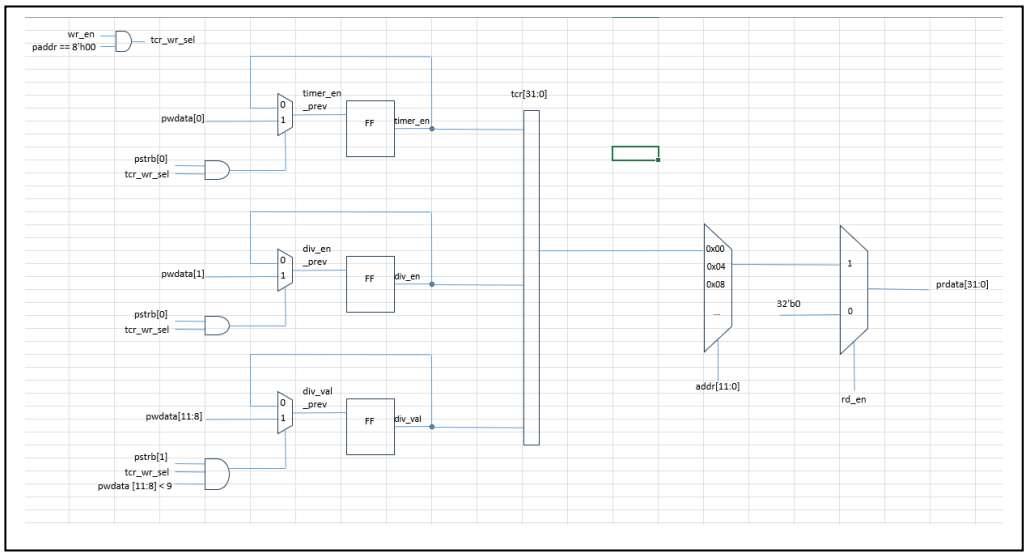

Figure 4. TCR Register logic diagram

When the address is 0x0, wr_en is High and pslverr is Low.

If tim_pstrb[0] is High, then:

- timer_en is stored with wdata[0].
- div_en is stored with wdata[1].

If tim_pstrb[1] is High, then:

- div_val is stored with wdata[11:8]

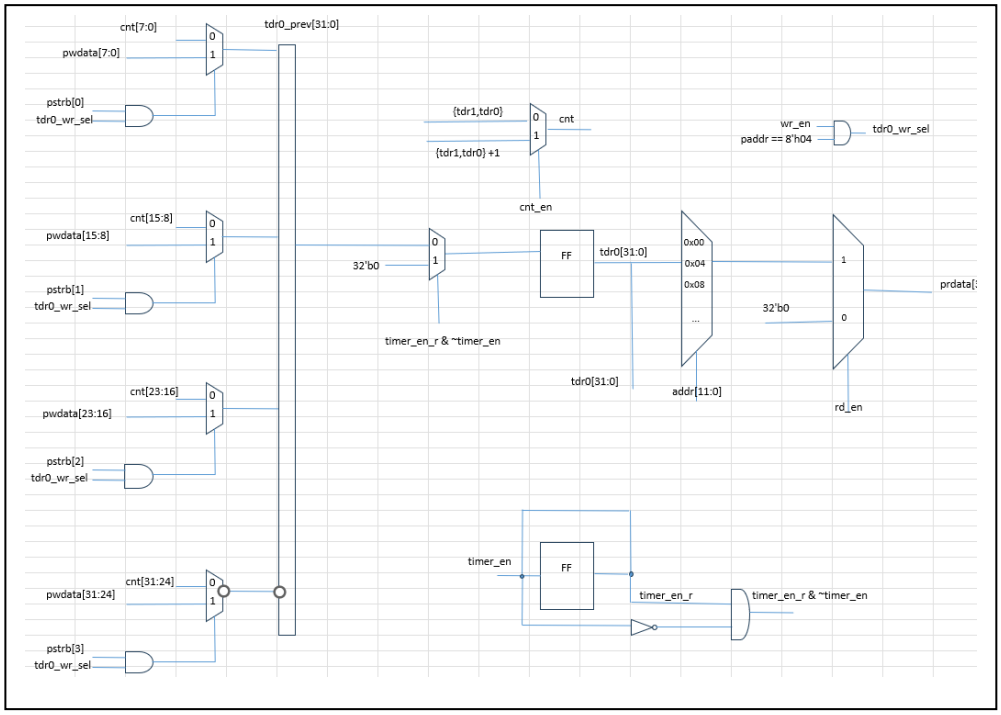

Figure 5. TDR0 Register logic diagram

cnt[63:0] is a 64-bit signal formed by combining two registers, tdr0 and tdr1.

Its value increments by one whenever cnt_en is asserted

When the address is 0x4 and wr_en is high.

- tim_pstrb[0] is high, a write access to byte 0 occurs, and TDR0[7:0] is updated with wdata[7:0].
- tim_pstrb[1] is high, a write access to byte 1 occurs, and TDR0[15:8] is updated with wdata[15:8].
- tim_pstrb[2] is high, a write access to byte 2 occurs, and TDR0[23:16] is updated with wdata[23:16].
- tim_pstrb[3] is high, a write access to byte 3 occurs, and TDR0[31:24] is updated with wdata[31:24].

timer_en_r is a flip-flop that stores the value of timer_en.

~timer_en & timer_en_r condition has the highest priority and resets TDR0 to 32'h0000_0000.

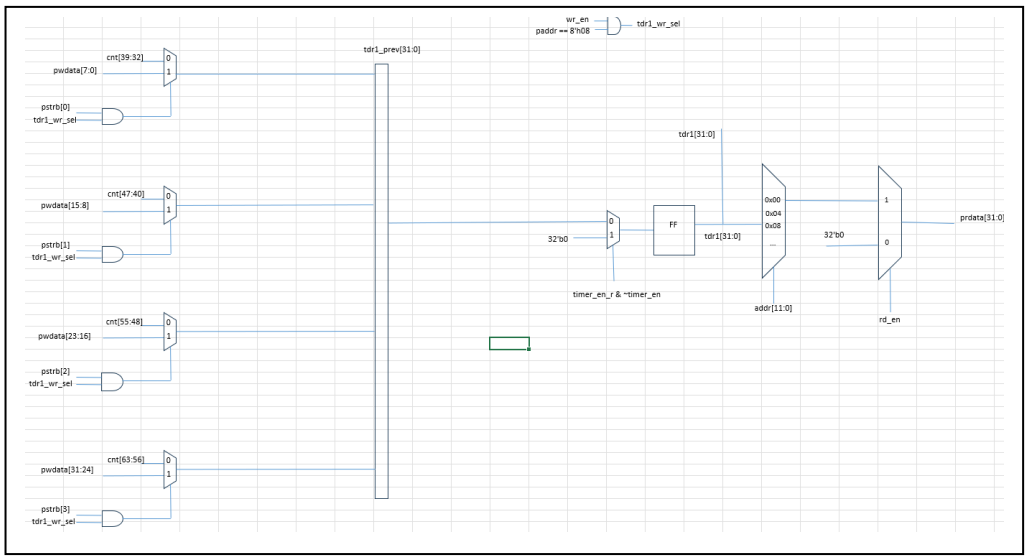

Figure 6. TDR1 Register logic diagram

When the address is 0x8 and wr_en is high.

- tim_pstrb[0] is high, a write access to byte 0 occurs, and TDR1[7:0] is updated with wdata[7:0].
- tim_pstrb[1] is high, a write access to byte 1 occurs, and TDR1[15:8] is updated with wdata[15:8].
- tim_pstrb[2] is high, a write access to byte 2 occurs, and TDR1[23:16] is updated with wdata[23:16].
- tim_pstrb[3] is high, a write access to byte 3 occurs, and TDR1[31:24] is updated with wdata[31:24].

~timer_en & timer_en_r condition has the highest priority and resets TDR1 to 32'h0000_0000.

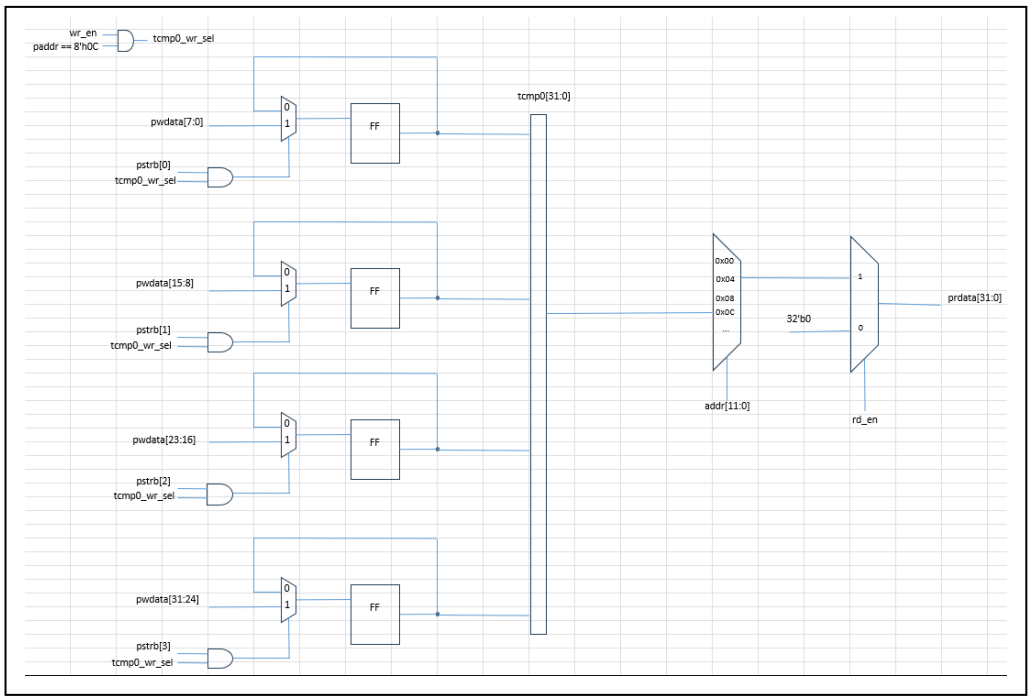

Figure 7. TCMP0 Register logic diagram

When the address is 0xC and wr_en is high.

- tim_pstrb[0] is high, a write access to byte 0 occurs, and TCMP0[7:0] is updated with wdata[7:0].
- tim_pstrb[1] is high, a write access to byte 1 occurs, and TCMP0[15:8] is updated with wdata[15:8].
- tim_pstrb[2] is high, a write access to byte 2 occurs, and TCMP0[23:16] is updated with wdata[23:16].
- tim_pstrb[3] is high, a write access to byte 3 occurs, and TCMP0[31:24] is updated with wdata[31:24].

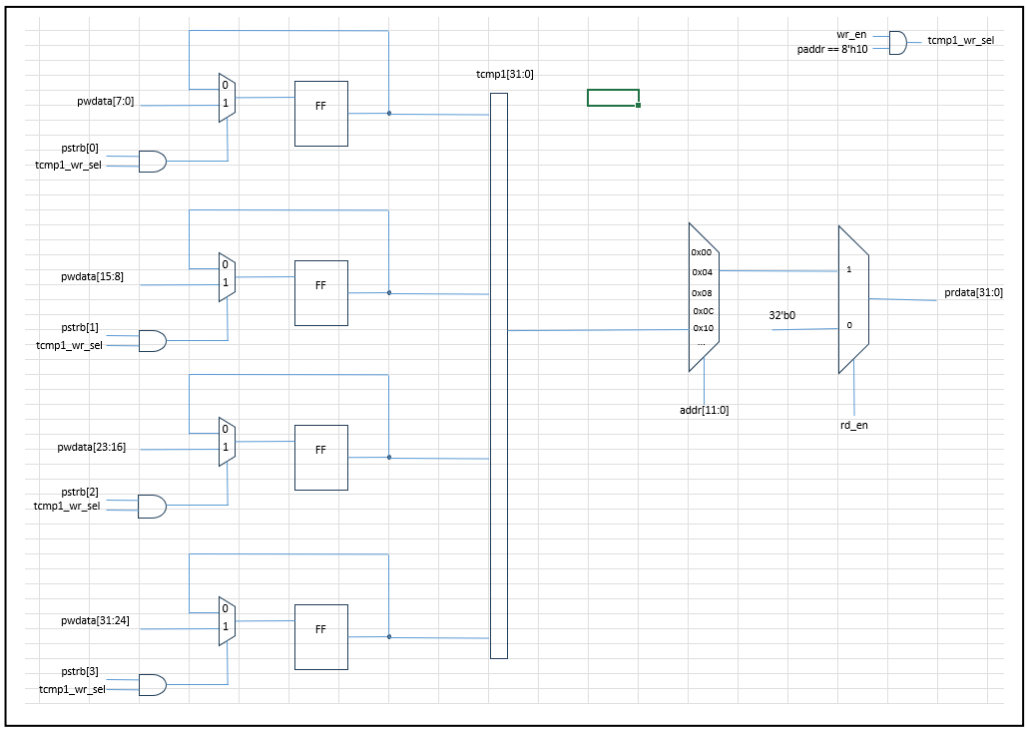

Figure 8. TCMP1 Register logic diagram

When the address is 0x10 and wr_en is high.

- tim_pstrb[0] is high, a write access to byte 0 occurs, and TCMP1[7:0] is updated with wdata[7:0].
- tim_pstrb[1] is high, a write access to byte 1 occurs, and TCMP1[15:8] is updated with wdata[15:8].
- tim_pstrb[2] is high, a write access to byte 2 occurs, and TCMP1[23:16] is updated with wdata[23:16].
- tim_pstrb[3] is high, a write access to byte 3 occurs, and TCMP1[31:24] is updated with wdata[31:24].

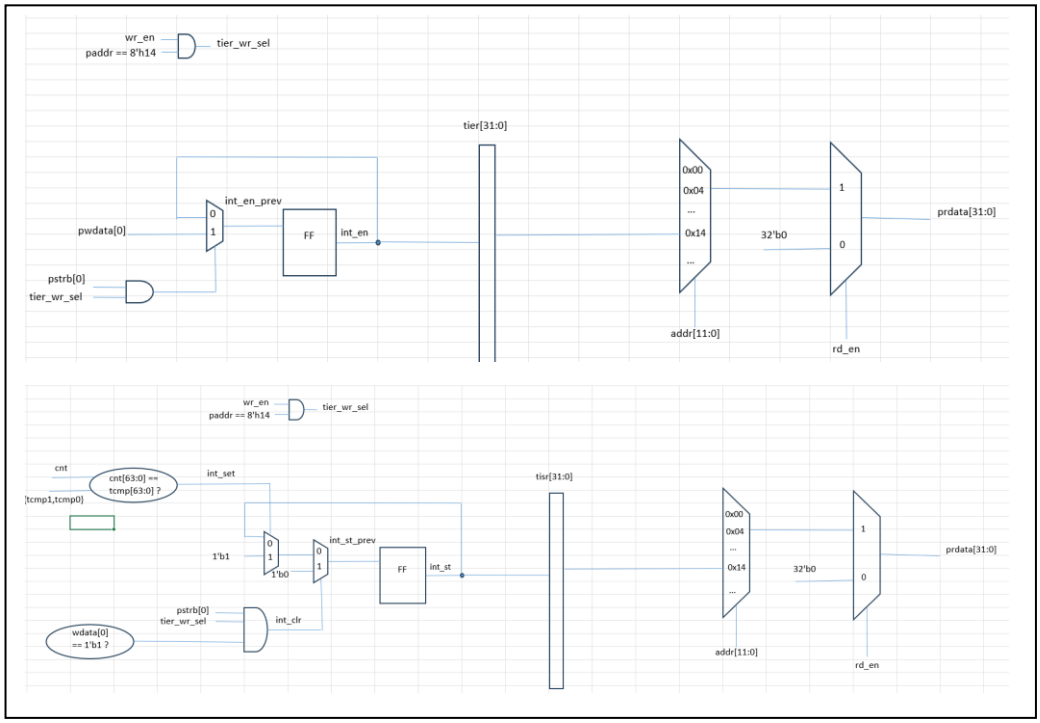

Figure 9. TIER and TISR Register logic diagram

When the address is 0x14 and wr_en is high.

- tim_pstrb[0] is high, a write access to byte 0 occurs, and int_en is updated with wdata[0].

When the address is 0x18 and wr_en is high.

- tim_pstrb[0] is high and wdata[0] is 1, int_clr becomes high, which clears int_st to 1'b0.
- tcmp is equal to the cnt value, int_set is asserted, int_st is updated with int_set.
- Clear has higher priority than Set for int_st.

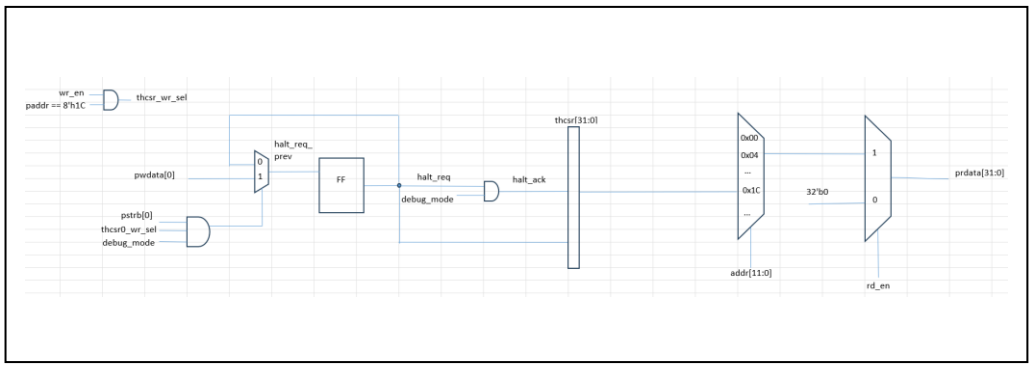

Figure 10. THCSR Register logic diagram

When the address is 0x1C and wr_en is high.

- tim_pstrb[0] is high, a write access to byte 0 occurs, and halt_req is updated with wdata[0].

halt_ack is asserted when both dbg_mode and halt_req are high.

3.2. Counter Control Combines With Halted, And Counting Mode

Counter Control:

- Including logic that generates cnt_en for the counter. It supports two main modes: the default mode and the div_val mode.

Counting mode:

- In default mode, counting speed depend on system clock (same as div_val=0 case).
- The counting's speed can be controlled by register settings by setting into TCR.div_en and TCR.div_val[3:0] as the register specification
- div_en and div_val can not be changed during timer_en is High.
- Not allow to change div_en and div_val while timer_en is High by an error response

Halted mode:

- Counter can be halted (stopped) in debug mode when both below conditions occur:
  - Input debug_mode signal is High,
  - THCR.halt_req is 1.
- THCSR.halt_ack is 1 after a halt request indicates that the request is accepted.
- After halted, counter can be resumed to count normally when clearing the halt request to 0.

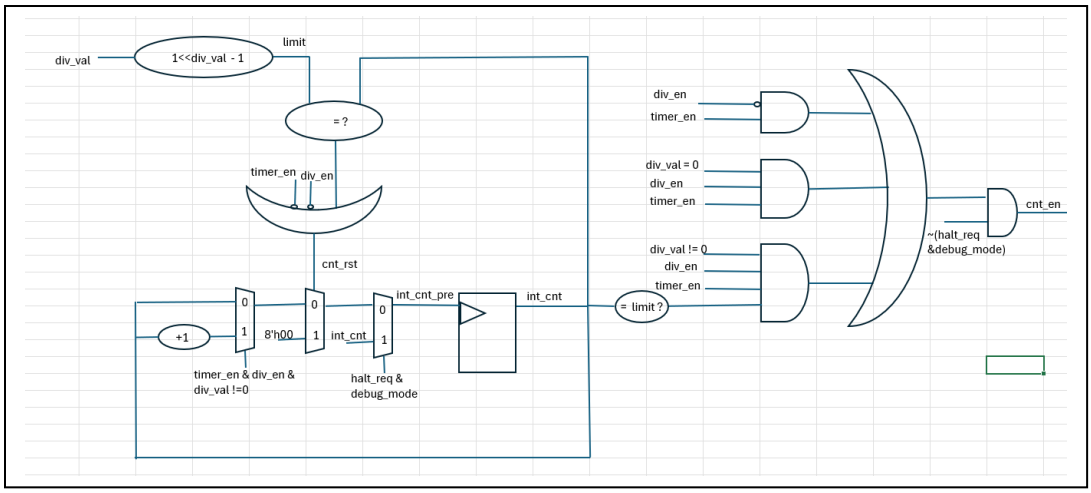

Figure 11. Counter Control logic diagram

- limit is the maximum value determined by the number of bits in div_val.
- int_cnt is used in div_val mode to determine after how many cycles cnt_en is asserted. When int_cnt_en is high (div_val != 0, div_en = 1'b1, and timer_en = 1'b1), int_cnt increments from 0 up to the value equal to limit.
- cnt_rst is used to clear int_cnt when any of the following three conditions occurs:
  - timer_en is disabled.
  - div_en is disabled.
  - int_cnt reaches the limit value.
- cnt_en_2 is asserted when div_en = 1'b0 and timer_en = 1'b1.
- cnt_en_1 is asserted when div_en = 1'b1, timer_en = 1'b1, div_val !=0 and int_cnt is matched limit .
- cnt_en_3 is asserted when div_en = 1'b1, timer_en = 1'b1, div_val ==0.
- When any of cnt_en_1, cnt_en_2, or cnt_en_3 is asserted, cnt_en is driven High.
- When dbg_mode = 1'b1 and halt_req = 1'b1, the halted state is enable:
  - Stop counting int_cnt.
  - cnt_en is always low.

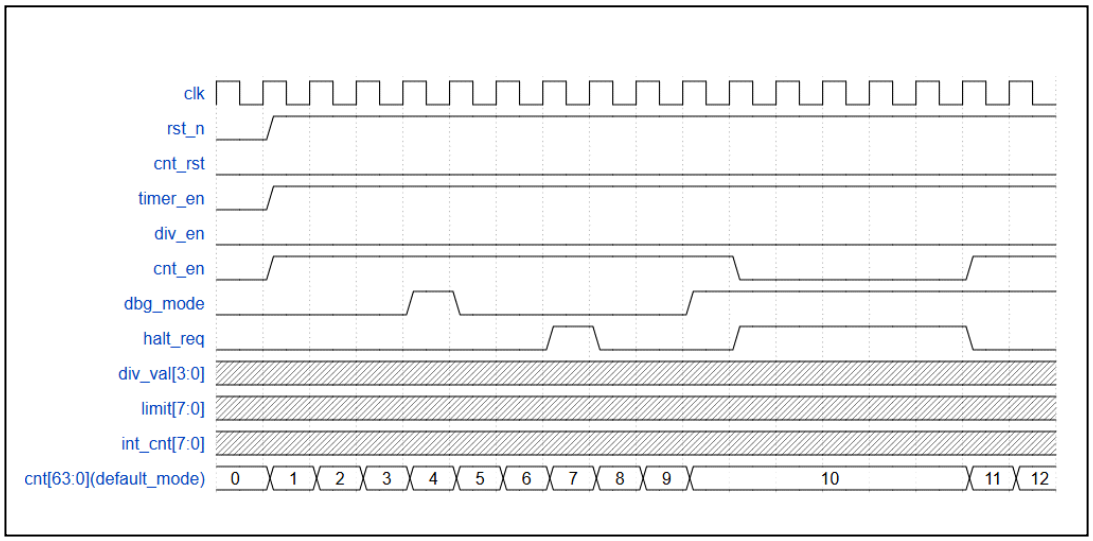

Figure 12. Waveform of the counter counting in default mode

The signal cnt_en is created when timer_en = 1'b1, div_en = 1'b0:

- cnt_en = 1, cnt increases one.
- cnt_en = 0, cnt is not changed.

Do not care about div_val, limit, int_cnt, and cnt_rst in this case.

When cnt = 10, then dbg_mode = 1'b1, halt_req = 1'b1, counter holds its value at 10.

When halt_req = 1'b0, counter continues counting value from 10.

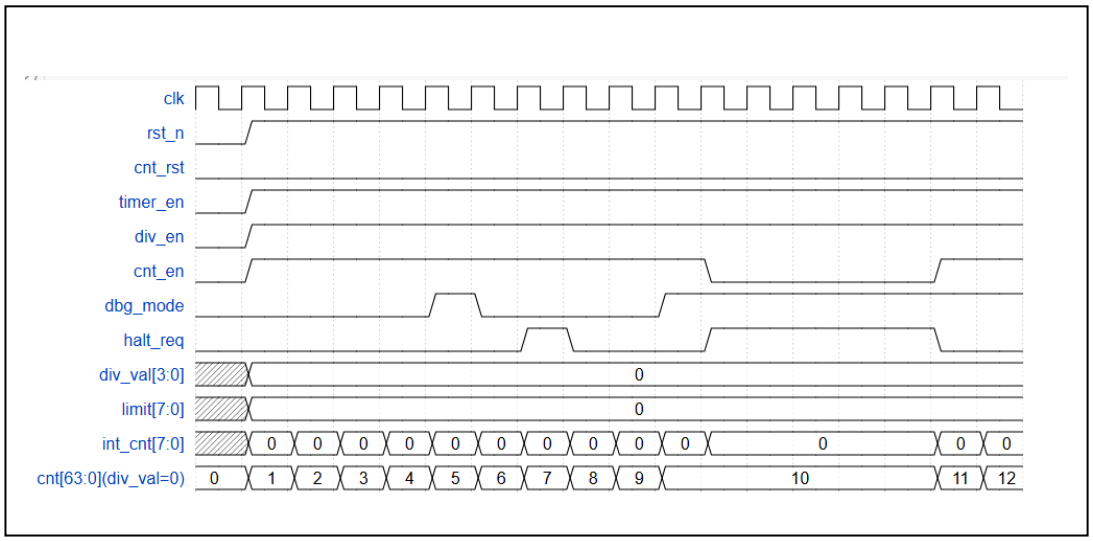

Figure 13. Waveform of the counter counting in mode div_val = 0

The signal cnt_en is created when timer_en = 1'b1, div_en = 1'b1, and div_val = 0:

- cnt_en = 1, cnt increases one.
- cnt_en = 0, cnt is not changed.

Do not care about limit, int_cnt, and cnt_rst in this case.

When cnt = 10, then dbg_mode = 1'b1, halt_req = 1'b1, counter holds its value at 10.

When halt_req = 1'b0, counter continues counting value from 10.

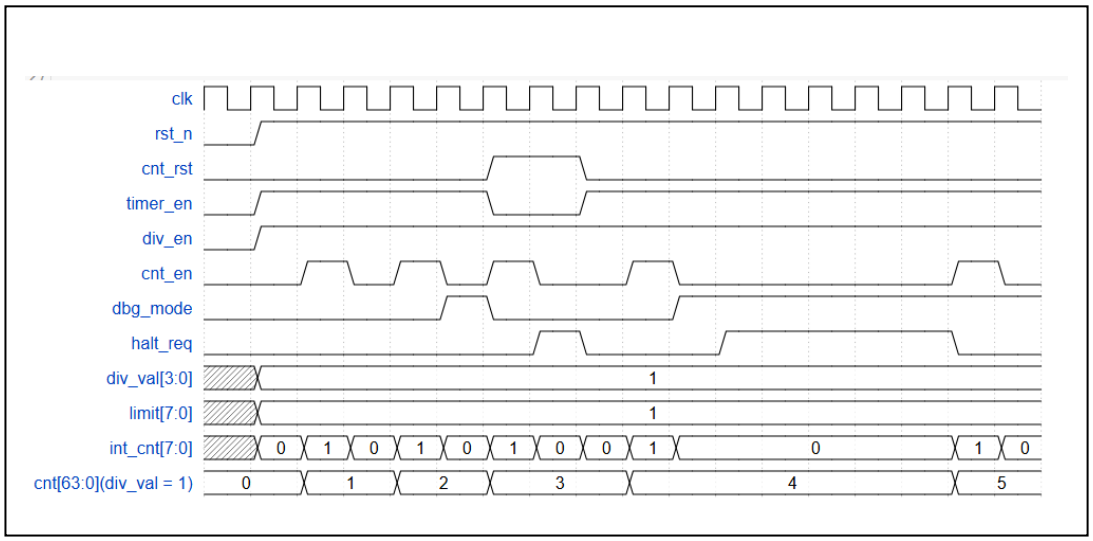

Figure 14. Waveform of the counter counting in mode div_val = 1

The signal cnt_en is created when timer_en = 1’b1, div_en = 1’b1, and div_val = 1:

- cnt_en = 1, cnt increases one after 2 cycles.
- cnt_en = 0, cnt is not changed.

cnt_rst is 1'b1 for two consecutive cycles, and int_cnt is cleared to 0 after one cycle of cnt_rst being high.

When cnt = 4, then dbg_mode = 1’b1, halt_req = 1’b1:

- counter holds its value at 4.
- int_cnt holds its value at 0.

When halt_req = 1’b0:

- int_cnt counts up to 1, the counter value reaches 5, and it continues operating.

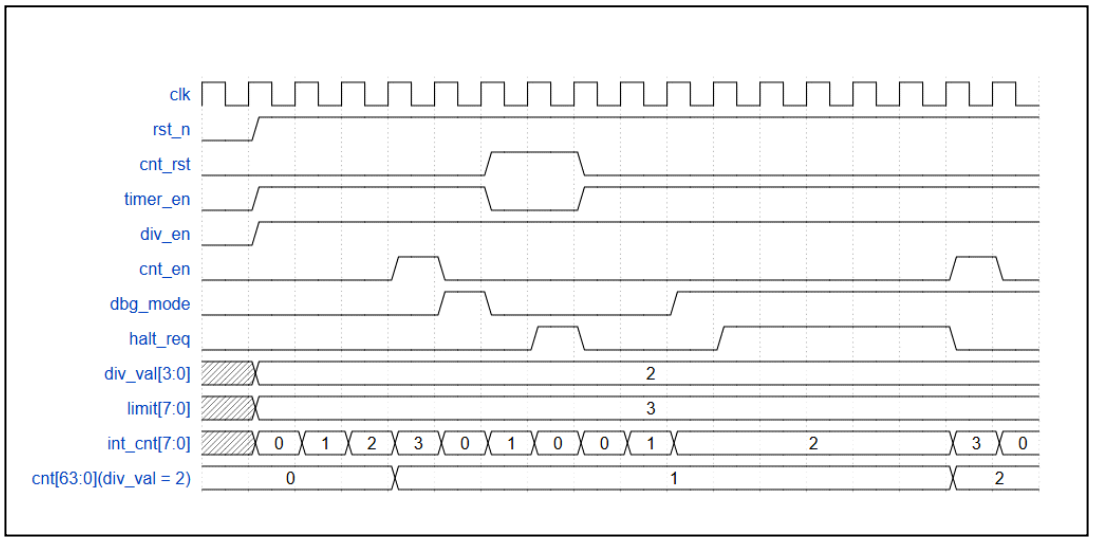

Figure 15. Waveform of the counter counting in mode div_val = 2

The signal cnt_en is created when timer_en = 1’b1, div_en = 1’b1, and div_val = 2:

- cnt_en = 1, cnt increases one after 4 cycles.
- cnt_en = 0, cnt is not changed.

cnt_rst is 1'b1 for two consecutive cycles, and int_cnt is cleared to 0 after one cycle of cnt_rst being high.

When cnt = 1, then dbg_mode = 1’b1, halt_req = 1’b1:

- counter holds its value at 1.
- int_cnt holds its value at 2.

When halt_req = 1’b0:

- int_cnt counts up to 3, the counter value reaches 2, and it continues operating.

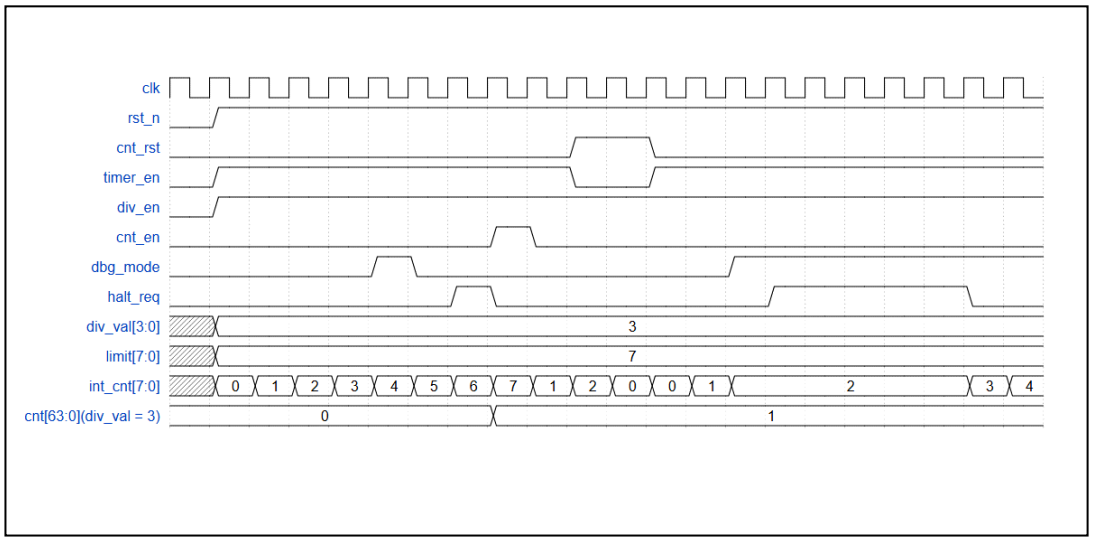

Figure 16. Waveform of the counter counting in mode div_val = 3

The signal cnt_en is created when timer_en = 1’b1, div_en = 1’b1, and div_val = 3:

- cnt_en = 1, cnt increases one after 8 cycles.
- cnt_en = 0, cnt is not changed.

cnt_rst is 1'b1 for two consecutive cycles, and int_cnt is cleared to 0 after one cycle of cnt_rst being high.

When cnt = 1, then dbg_mode = 1’b1, halt_req = 1’b1:

- counter holds its value at 1.
- int_cnt holds its value at 2.

When halt_req = 1’b0:

- int_cnt counts up to 3, the counter continues operating.

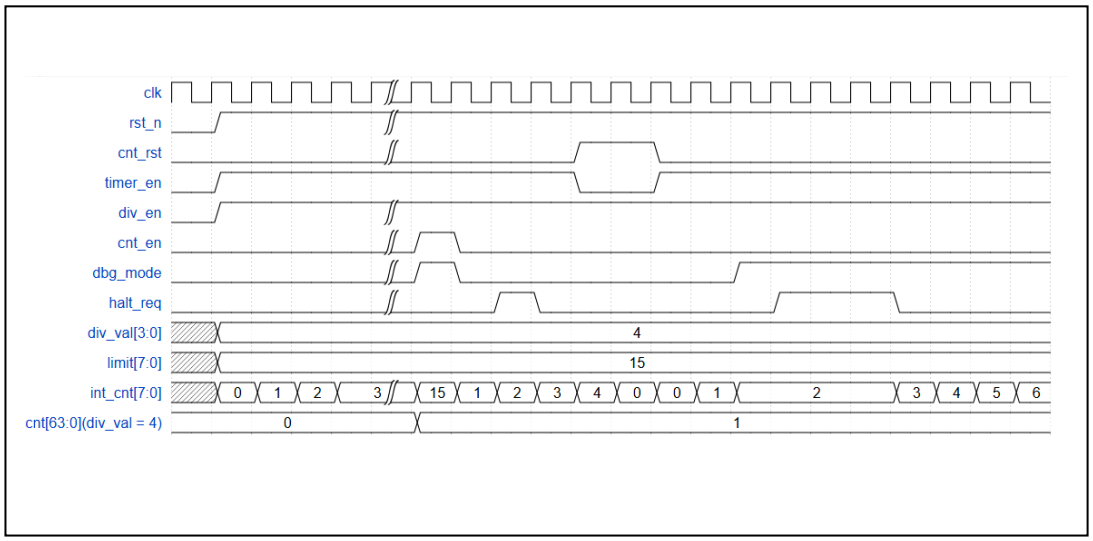

Figure 17. Waveform of the counter counting in mode div_val = 4

The signal cnt_en is created when timer_en = 1’b1, div_en = 1’b1, and div_val = 4:

- cnt_en = 1, cnt increases one after 16 cycles.
- cnt_en = 0, cnt is not changed.

cnt_rst is 1'b1 for two consecutive cycles, and int_cnt is cleared to 0 after one cycle of cnt_rst being high.

When cnt = 1, then dbg_mode = 1’b1, halt_req = 1’b1:

- counter holds its value at 1.
- int_cnt holds its value at 2.

When halt_req = 1’b0:

- int_cnt counts up to 3, the counter continues operating.

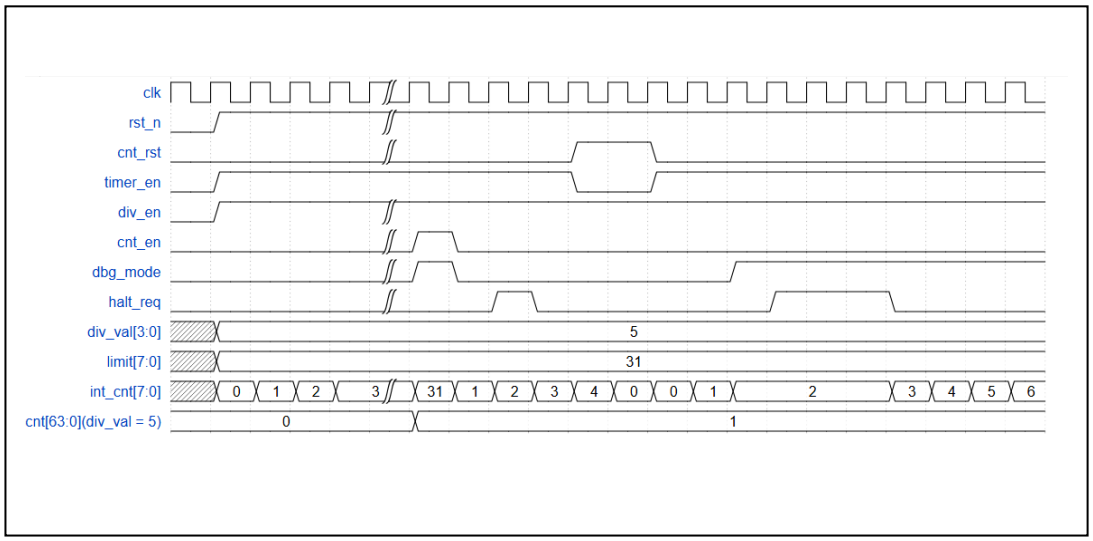

Figure 18. Waveform of the counter counting in mode div_val = 5

The signal cnt_en is created when timer_en = 1’b1, div_en = 1’b1, and div_val = 5:

- cnt_en = 1, cnt increases one after 32 cycles.
- cnt_en = 0, cnt is not changed.

cnt_rst is 1'b1 for two consecutive cycles, and int_cnt is cleared to 0 after one cycle of cnt_rst being high.

When cnt = 1, then dbg_mode = 1’b1, halt_req = 1’b1:

- counter holds its value at 1.
- int_cnt holds its value at 2.

When halt_req = 1’b0:

- int_cnt counts up to 3, the counter continues operating.

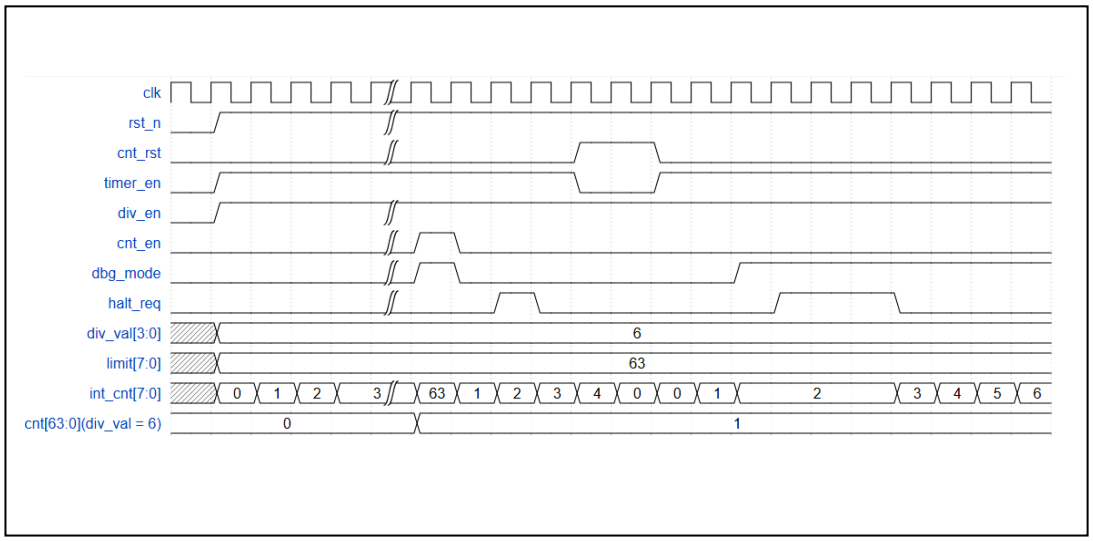

Figure 19. Waveform of the counter counting in mode div_val = 6

The signal cnt_en is created when timer_en = 1’b1, div_en = 1’b1, and div_val = 6:

- cnt_en = 1, cnt increases one after 64 cycles.
- cnt_en = 0, cnt is not changed.

cnt_rst is 1'b1 for two consecutive cycles, and int_cnt is cleared to 0 after one cycle of cnt_rst being high.

When cnt = 1, then dbg_mode = 1’b1, halt_req = 1’b1:

- counter holds its value at 1.
- int_cnt holds its value at 2.

When halt_req = 1’b0:

- int_cnt counts up to 3, the counter continues operating.

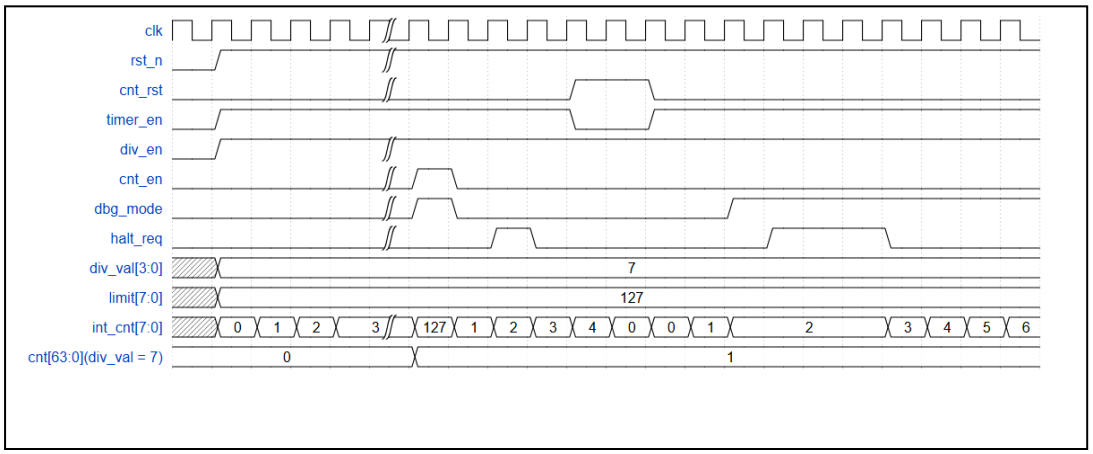

Figure 20. Waveform of the counter counting in mode div_val = 7

The signal cnt_en is created when timer_en = 1’b1, div_en = 1’b1, and div_val = 7:

- cnt_en = 1, cnt increases one after 128 cycles.
- cnt_en = 0, cnt is not changed.

cnt_rst is 1'b1 for two consecutive cycles, and int_cnt is cleared to 0 after one cycle of cnt_rst being high.

When cnt = 1, then dbg_mode = 1’b1, halt_req = 1’b1:

- counter holds its value at 1.
- int_cnt holds its value at 2.

When halt_req = 1’b0:

- int_cnt counts up to 3, the counter continues operating.

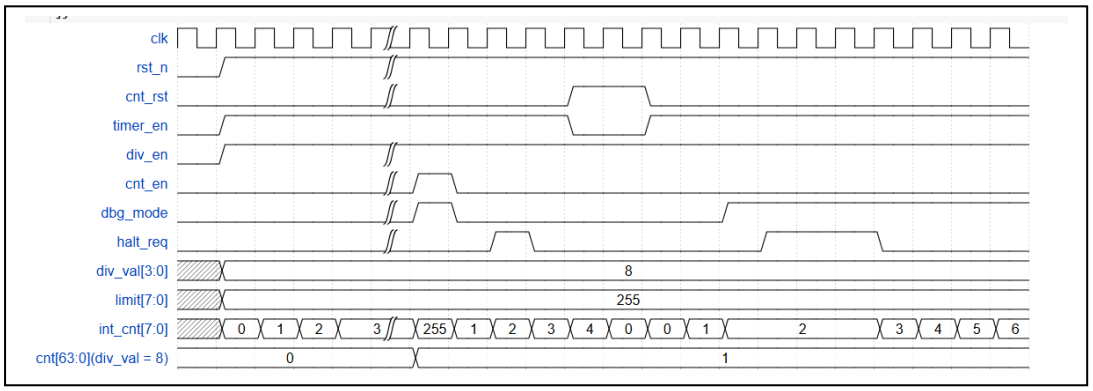

Figure 21. Waveform of the counter counting in mode div_val = 8

The signal cnt_en is created when timer_en = 1'b1, div_en = 1'b1, and div_val = 8:

- cnt_en = 1, cnt increases one after 256 cycles.
- cnt_en = 0, cnt is not changed.

cnt_rst is 1'b1 for two consecutive cycles, and int_cnt is cleared to 0 after one cycle of cnt_rst being high.

When cnt = 1, then dbg_mode = 1'b1, halt_req = 1'b1:

- counter holds its value at 1.
- int_cnt holds its value at 2.

When halt_req = 1'b0:

- int_cnt counts up to 3, the counter continues operating.

3.3. Timer Interrupt

- Timer interrupt (tim_int) is asserted (set) when interrupt is enabled and counter's value matches (equal) the compare value.
- Once asserted, the timer interrupt (tim_int) remains unchange until it is cleared by writing 1 to TISR.int_st bit or the interrupt is disabled.

4. Report

RTL Codeding:

Path: ictc/student_data/dangminh_0402/final_project_m/rtl

Testbench Coding:

Path: ictc/student_data/dangminh_0402/final_project_m /tb

V-PLAN:

Table 11. V-plan for verification

ID	ITEM	TEST_SEQUENCE	PASS_CONDITION
1	rst_chk	Sys_rst_n = 0	ADDR_TCR: prdata = 32'h0000_0100 ADDR_TDR0: prdata = 32'h0000_0000 ADDR_TDR1: prdata = 32'h0000_0000 ADDR_TCMP0: prdata = 32'hFFFF_FFFF ADDR_TCMP1: prdata = 32'hFFFF_FFFF ADDR_TIER: prdata = 32'h0000_0000 ADDR_TISR: prdata = 32'h0000_0000 ADDR_THCSR: prdata = 32'h0000_0000
2	rd_wr_chk	ADDR_TCR Write 0000_0000, ffff_ffff, 5555_5555, aaaa_aaaa, then read and compare prdata with expected values	Pwdata = 0000_0000: prdata = 0000_0000 Pwdata = 3333_3333 prdata = 0000_0303 Pwdata = aaaa_aaaa: prdata = 0000_0303 Pwdata = ffff_ffff: prdata = 0000_0303
		ADDR_TDR0 Write 0000_0000, ffff_ffff, 5555_5555, aaaa_aaaa, then read and compare prdata with expected values	Pwdata = 0000_0000: prdata = 0000_0000 Pwdata = ffff_ffff: prdata = ffff_ffff Pwdata = 5555_5555: prdata = 5555_5555 Pwdata = aaaa_aaaa: prdata = aaaa_aaaa
		ADDR_TDR1 Write 0000_0000, ffff_ffff, 5555_5555, aaaa_aaaa, then read and compare prdata with expected values	Pwdata = 0000_0000: prdata = 0000_0000 Pwdata = ffff_ffff: prdata = ffff_ffff Pwdata = 5555_5555: prdata = 5555_5555 Pwdata = aaaa_aaaa: prdata = aaaa_aaaa
		ADDR_TCMP0 Write 0000_0000, ffff_ffff, 5555_5555, aaaa_aaaa, then read and compare prdata with expected values	Pwdata = 0000_0000: prdata = 0000_0000 Pwdata = ffff_ffff: prdata = ffff_ffff Pwdata = 5555_5555: prdata = 5555_5555 Pwdata = aaaa_aaaa: prdata = aaaa_aaaa
		ADDR_TCMP1 Write 0000_0000, ffff_ffff, 5555_5555, aaaa_aaaa, then read and compare prdata with expected values	Pwdata = 0000_0000: prdata = 0000_0000 Pwdata = ffff_ffff: prdata = ffff_ffff Pwdata = 5555_5555: prdata = 5555_5555 Pwdata = aaaa_aaaa: prdata = aaaa_aaaa
		ADDR_TIER	Pwdata = 0000_0000: prdata = 0000_0000 Pwdata = 3333_3333: prdata = 0000_0001

		Write 0000_0000, ffff_ffff, 5555_5555, aaaa_aaaa, then read and compare prdata with expected values	Pwdata = aaaa_aaaa: prdata = 0000_0000 Pwdata = ffff_ffff: prdata = 0000_0001
		ADDR_TISR Write 0000_0000, ffff_ffff, 5555_5555, aaaa_aaaa, then read and compare prdata with expected values	Pwdata = 0000_0000: prdata = 0000_0000 Pwdata = 3333_3333: prdata = 0000_0000 Pwdata = aaaa_aaaa: prdata = 0000_0000 Pwdata = ffff_ffff: prdata = 0000_0000
		ADDR_THCSR Write 0000_0000, ffff_ffff, 5555_5555, aaaa_aaaa, then read and compare prdata with expected values	Pwdata = 0000_0000: prdata = 0000_0000 Pwdata = ffff_ffff: prdata = 0000_0001 Pwdata = 5555_5555: prdata = 0000_0001 Pwdata = aaaa_aaaa: prdata = 0000_0000
3	Cnt_ctrl & halt	Div_en is off	Tdr0 = 0000_0000, tdr1 = 0000_0000 TCR = 0000_0001 Wait 100 cycle THCSR = 0000_0001 to halt ➔ Tdr0 = 0000_0068 ➔ Tdr1 = 0000_0000 Tdr0 = FFFF_FF00, tdr1 = 0000_0000 TCR = 0000_0001 Wait 256 cycle THCSR = 0000_0001 to halt ➔ Tdr0 = 0000_0004 ➔ Tdr1 = 0000_0001
		Div_en is on	Div_val = 0000, wait 346 cycle and halt ➔ Tdr0 = 0000_015e ➔ Tdr1 = 0000_0000 Div_val = 0001, wait 256 cycle and halt ➔ Tdr0 = 0000_0082 ➔ Tdr1 = 0000_0000 Div_val = 0010, wait 256 cycle and halt ➔ Tdr0 = 0000_0041 ➔ Tdr1 = 0000_0000 Div_val = 0011, wait 256 cycle and halt ➔ Tdr0 = 0000_0020 ➔ Tdr1 = 0000_0000 Div_val = 0100, wait 256 cycle and halt ➔ Tdr0 = 0000_0010 ➔ Tdr1 = 0000_0000

			Div_val = 0101, wait 256 cycle and halt ➔ Tdr0 = 0000_0008 ➔ Tdr1 = 0000_0000 Div_val = 0110, wait 256 cycle and halt ➔ Tdr0 = 0000_0004 ➔ Tdr1 = 0000_0000 Div_val = 0111, wait 256 cycle and halt ➔ Tdr0 = 0000_0002 ➔ Tdr1 = 0000_0000 Div_val = 1000, wait 256 cycle and halt ➔ Tdr0 = 0000_0001 ➔ Tdr1 = 0000_0000
		Div_en is off	Div_val = 1000, wait 100 cycle and halt ➔ Tdr0 = 0000_0068 ➔ Tdr1 = 0000_0000
4	Apb_pslverr_chk	Change div_val when timer_en assert	Write 000_0103 to TCR Write 0000_0203to TCR Then check pslverr Write 000_0203 to TCR Write 0000_0503to TCR Then check pslverr
		Div_val > 4'b1000	Write pwdata [11:8] > 1000 Then check plsverr
		Change div_en when timer_en assert	Write 000_0003 to TCR Write 0000_0001to TCR Then check pslverr
5	Check_reserved_addr	Write ffff_ffff to 0x20, 0x28, … and check prdata	Paddr = 0x20: prdata = 0000_0000 Paddr = 0x24: prdata = 0000_0000 Paddr = 0x28: prdata = 0000_0000 Paddr = 0x80: prdata = 0000_0000 Paddr = 0x84: prdata = 0000_0000
6	byte_access_chk	ADDR_TCR pwdata = FFFF_FFFF	pstrb = 0001: prdata = 32'h0000_0103 pstrb = 0010: prdata = 32'h0000_0103 pstrb = 0100: prdata = 32'h0000_0103 pstrb = 1000: prdata = 32'h0000_0103
		ADDR_TDR0 pwdata = FFFF_FFFF	pstrb = 0001: prdata = 32'h0000_00FF pstrb = 0010: prdata = 32'h0000_FFFF pstrb = 0100: prdata = 32'h00FF_FFFF pstrb = 1000: prdata = 32'hFFFF_FFFF
		ADDR_TDR1 pwdata = FFFF_FFFF	pstrb = 0001: prdata = 32'h0000_00FF pstrb = 0010: prdata = 32'h0000_FFFF

			pstrb = 0100: prdata = 32'h00FF_FFFF pstrb = 1000: prdata = 32'hFFFF_FFFF
		ADDR_TCMP0 pwdata = 0000_0000	pstrb = 0001: prdata = 32'hFFFF_FF00 pstrb = 0010: prdata = 32'hFFFF_0000 pstrb = 0100: prdata = 32'hFF00_0000 pstrb = 1000: prdata = 32'h0000_0000
		ADDR_TCMP1 pwdata = 0000_0000	pstrb = 0001: prdata = 32'hFFFF_FF00 pstrb = 0010: prdata = 32'hFFFF_0000 pstrb = 0100: prdata = 32'hFF00_0000 pstrb = 1000: prdata = 32'h0000_0000
		ADDR_TIER pwdata = FFFF_FFFF	pstrb = 0001: prdata = 32'h0000_0001 pstrb = 0010: prdata = 32'h0000_0001 pstrb = 0100: prdata = 32'h0000_0001 pstrb = 1000: prdata = 32'h0000_0001
		ADDR_TISR pwdata = FFFF_FFFF	Write data 0000_0001 at addr 0x14 Write data 0000_000a at addr 0x0c Write data 0000_0000 at addr 0x10 Write data 0000_0000 at addr 0x08 Write data 0000_000a at addr 0x04 ➔ Tim_init assert pstrb = 0010: prdata = 32'h0000_0001 pstrb = 0100: prdata = 32'h0000_0001 pstrb = 1000: prdata = 32'h0000_0001 Write data 0000_0000 at addr 0x0c Write data 0000_0000 at addr 0x10 pstrb = 0001: prdata = 32'h0000_0000 ➔ Tim_init deassert
		ADDR_THCSR pwdata = FFFF_FFFF	pstrb = 0001: prdata = 32'h0000_0001 pstrb = 0010: prdata = 32'h0000_0001 pstrb = 0100: prdata = 32'h0000_0001 pstrb = 1000: prdata = 32'h0000_0001
7	1hot_chk	Write 0000_0000 to TCR Write 1111_1111 to TDR0 Write 2222_2222 to TDR1 Write 3333_3333 to TCMP0 Write 4444_4444 to TCMP1 Write 5555_5555 to TISR Write 6666_6666 to TIER Write 7777_7777 to THCSR Read prdata and compare with expected value	TCR: prdata = 0000_0000 TDR0: prdata = 1111_1111 TDR1: prdata = 2222_2222 TCMP0: prdata = 3333_3333 TCMP1: prdata = 4444_4444 TIER: prdata = 0000_0001 TISR: prdata = 0000_0000 THCSR: prdata = 0000_0001

Testcase :

Path: ictc/student_data/dangminh_0402/final_project_m /testcases

```
dangminh_0402@ictc-eda-ldap:~/final_project_m/testcases$ ll
total 60
drwxrwxr-x+ 2 dangminh_0402 dangminh_0402 4096 Thg 12 14 11:28 ./
drwxrwxr-x+ 6 dangminh_0402 dangminh_0402 4096 Thg 12  6 13:09 ../
-rw-rw-r--+ 1 dangminh_0402 dangminh_0402 2913 Thg 11 30 12:56 testcase_1_hot_check.v
-rw-rw-r--+ 1 dangminh_0402 dangminh_0402 4217 Thg 11 30 12:49 testcase_addr_reserved_chk.v
-rw-rw-r--+ 1 dangminh_0402 dangminh_0402 2782 Thg 12  5 17:37 testcase_apb_pslverr.v
-rw-rw-r--+ 1 dangminh_0402 dangminh_0402 8931 Thg 12  5 14:21 testcase_byte_access_chk.v
-rw-rw-r--+ 1 dangminh_0402 dangminh_0402 9412 Thg 12 12 13:42 testcase_cnt_ctrl.v
-rw-rw-r--+ 1 dangminh_0402 dangminh_0402 5263 Thg 12  5 14:10 testcase_rd_wr_chk.v
-rw-rw-r--+ 1 dangminh_0402 dangminh_0402 2059 Thg 11 30 12:49 testcase_rst_chk.v
```

Figure 22. Testcases list

Testcases report in simulation:

```
dangminh_0402@ictc-eda-ldap:~/final_project_m/sim$ ./report.csh
testcase_rst_chk
testcase_rd_wr_chk
testcase_addr_reserved_chk
testcase_byte_access_chk
testcase_1_hot_check
testcase_cnt_ctrl
testcase_apb_pslverr
|-----------------------------------------------------------------------------------------|
| PAT_NAME                                | RUN_DATE                      | RESULT          |
|-----------------------------------------------------------------------------------------|
| testcase_rst_chk                        | 10:32:56 Jan 06 2026          | PASSED          |
|-----------------------------------------------------------------------------------------|
| testcase_rd_wr_chk                      | 10:32:58 Jan 06 2026          | PASSED          |
|-----------------------------------------------------------------------------------------|
| testcase_addr_reserved_chk              | 10:33:01 Jan 06 2026          | PASSED          |
|-----------------------------------------------------------------------------------------|
| testcase_byte_access_chk                | 10:33:03 Jan 06 2026          | PASSED          |
|-----------------------------------------------------------------------------------------|
| testcase_1_hot_check                    | 10:33:05 Jan 06 2026          | PASSED          |
|-----------------------------------------------------------------------------------------|
| testcase_cnt_ctrl                       | 10:33:07 Jan 06 2026          | PASSED          |
|-----------------------------------------------------------------------------------------|
| testcase_apb_pslverr                    | 10:33:09 Jan 06 2026          | PASSED          |
|-----------------------------------------------------------------------------------------|
```

Figure 23. Test Cases Report

Coverage:

```
Coverage Report Summary Data by instance

================================================================================
=== Instance: /test_bench/u_dut/apb_slave
=== Design Unit: work.apb_slave_wait_state
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         4         4         0   100.00%
    Expressions                     16        16         0   100.00%
    Statements                      12        12         0   100.00%
    Toggles                         16        16         0   100.00%

================================================================================
=== Instance: /test_bench/u_dut/register
=== Design Unit: work.register_module
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        92        92         0   100.00%
    Conditions                      37        37         0   100.00%
    Expressions                     45        45         0   100.00%
    Statements                     106       106         0   100.00%
    Toggles                        910       910         0   100.00%

================================================================================
=== Instance: /test_bench/u_dut/counter_control
=== Design Unit: work.cnt_ctrl
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         8         8         0   100.00%
    Conditions                       5         4         1    80.00%
    Expressions                     10         9         1    90.00%
    Statements                       7         7         0   100.00%
    Toggles                         50        50         0   100.00%

================================================================================
=== Instance: /test_bench/u_dut
=== Design Unit: work.timer_top
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                        156       156         0   100.00%

================================================================================
=== Instance: /test_bench
=== Design Unit: work.test_bench
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      18        18         0   100.00%
```

Figure 24. Coverage